

Project handover

The current local project, ready for the next team to inspect and run.

Snapshot

16 September 2026. This package contains the current working files, including local additions and changes that are not in the GitHub baseline. It is a source handover, not a production deployment. No claim is made that these changes have been pushed to GitHub.

What is delivered

The main Node/React application, original Yuzee prompt and JSON contracts, 103 updated user-facing skill instructions, routing and research code, tests, optional separate microtools concept, Java backend source, HTML review reports and selected synthetic response records. `START_HERE.md` and `HANDOVER/AI_HANDOVER.md` explain how to continue.

Current integration

The improved counselling guidance is used by the main Quiz/Pathway chat. A message explicitly addressed to @Oala can receive one MiniLM-selected skill instruction. The application adds that instruction alongside the user input, then Gemini writes the answer using retained conversation context. Ordinary Quiz submissions do not invoke MiniLM.

Not yet automatic

There is no conversation-wide skill planner, no execution of all registry next-tool links, and no automatic evidence retrieval just because a skill is selected. The separate Explore more action provides a scoped research flow. Skill selection is a topic hint, not proof of facts or permission to perform a service.

Package boundaries

API keys, .env files, database settings, private chat history, runtime logs, Git history, installed dependencies, compiled output and model weights are excluded. The receiving team supplies its own credentials and installs dependencies. Existing runtime data is not needed to start the project.

Code baseline

Git baseline: 8d5ea0e9db39c843ca383fcc37adb4be37671709. The archive includes newer local working-tree changes. `HANDOVER/SNAPSHOT.json` records the baseline and changed paths; `MANIFEST.sha256` records the delivered files.

What changed

Changes are connected to the main chat; the separate concept is clearly identified.

Counselling and depth

Shared guidance now favours a direct answer, plain explanations, useful reasoning and worked examples. Depth follows the question: a short eligibility check need not become a lesson, while a learning request should not collapse into one generic paragraph. Examples, practice and sample answers are optional teaching ingredients, not compulsory repeated headings.

103 individual skill instructions

The checked-in registry has 105 tool records. CORE_001 and CORE_002 are internal, leaving 103 user-facing routing candidates. All 103 have individual reviewed guidance. Earlier conversation references to 106 triggers do not match the current tool count. The preserved concept registry also contains 40 rule records; these are not 40 additional executable tools.

Ten targeted refinements

Course units: worked examples and consequences. Entry evidence: explain a work sample. Costs: supplied subtotal and unknowns. Hiring difficulty: duplicates are not evidence of an unfilled role. Curriculum skills: distinguish explicit content from inference. Readiness: entering versus checking records. Prioritisation: conditional choices and alternatives. Pay: comparable inputs without invented hours. Clarification: weak/better dialogue and practice. Placement: task fit, supervision, capacity and approval.

Main output and interactions

The main renderer supports readable content blocks, comparisons and answer forms. Location is typed by the user. Interaction submissions are checked against the active server question. Copy and speech use visible content. Responsive output and scenario checks cover many content shapes and failure cases.

Streaming and review

Compact rotating status titles communicate routing, waiting, receiving, reviewing and checking. Internal thought text is not a user-facing transcript. Complete responses are validated before display. Eligible substantive structured answers receive an additional explanation review that may edit content blocks but cannot change interaction or service state.

Oala and persistence

@Oala has explicit per-message activation, product-owned service descriptions and fixed basic FAQ responses. Optional local routing falls back to the main counsellor. Local conversation storage uses serialized updates and atomic file replacement; the optional PostgreSQL path remains available.

How the main request works

MiniLM selects a task. Gemini produces the counselling response.

1. The user submits

Free text and structured Quiz answers enter `TokenLabContext.sendMessage`. A Quiz answer carries its question identifier and selected values or typed fields. The server rejects mismatched, stale or malformed interaction events.

2. Optional skill selection

Only an explicit current `@Oala` free-text message invokes MiniLM. It embeds the stripped current message, not the whole conversation. It compares that vector with short descriptors for 103 tasks. It does not embed all long mini-prompts or generate the final answer.

3. Confidence and fallback

The model is `Xenova/all-MiniLM-L6-v2` in a browser worker. Selection requires a cosine score of at least 0.48 and a lead of at least 0.06 over the next candidate. Guards skip structured answers, ambiguous short follow-ups, corrections, sensitive boundaries and other unsupported inputs. Cold loading never blocks chat; ready inference has a 1.5-second deadline.

4. The server adds approved guidance

The server checks the current mention, allowed tool ID and score bounds, and looks up its own mini-prompt. Browser-supplied prompt text is not accepted. The selected task guidance is appended to the current user input as optional focus. The original main prompt and canonical output contract retain control.

5. Gemini receives context

The request contains the main prompt plus shared clarity/depth rules; Oala service guidance when addressed; the current message or answer event; career context; retained dialogue and any summary within the memory budget; and available date, entered location, relevant user facts and prior question answers. A chosen skill is extra guidance, not a substitute for context.

6. Review, validation and display

Gemini returns the main protocol response. Eligible substantive structured answers get a second bounded Gemini review, then complete-response validation and the existing renderer. Review adds latency and usage. Basic Oala FAQs can bypass generation. The reviewer checks explanation against supplied context; it does not independently establish facts.

Evidence, limits and user experience

A deeper answer must help the person understand and decide.

Input limits are separate from answer limits

MiniLM input has a deliberate 256-token guard plus a character guard. Oversized input is skipped rather than silently truncated for routing; the full question can still reach Gemini. The selected mini-prompt is sent to Gemini and is not constrained by MiniLM's embedding budget. Gemini has separate context and output budgets.

Depth without forced length

Keep the direct answer visible, explain unfamiliar terms, show reasoning when useful and connect the result to the person's stated goal. Use tables only for comparisons, steps for processes and examples for learning. Do not invent details merely to fill a template. Explicit requests for brevity, corrections and stopping take priority.

Evidence must remain visible

Use plain distinctions such as Explicitly stated, Reasonably derived and Not enough information where useful. These are explanatory labels, not proof of truth or new schema enums. User statements remain self-reported. A certificate does not by itself prove skill proficiency; an elective does not prove everyone learns that skill.

Explore more is a different flow

The user supplies a scoped target and question, with year and location when relevant.

DetailResearchService first requests Google Search grounding, extracts eligible cited evidence, and then asks a specialist prompt to answer from that evidence. It validates the result, may attempt one bounded repair, and saves the result. It sends scoped research inputs, not the full profile/transcript, to search.

Research output states

Research distinguishes answered, partial, needs_clarification and no_evidence. Facts refer to actual evidence IDs; sources and gaps remain available beyond the short summary. An explicit refresh can fetch again. Saved research can be passed back to main chat by a conversation-scoped reference. This does not make every ordinary chat answer source-grounded.

Completion and honesty

Do not concatenate incomplete JSON into a complete-looking answer. Output limits, failed review, missing evidence and cancellation need a recoverable state. Unknown fees, availability, workload, eligibility and service execution stay unknown until supported. There are no connected booking, matching or application actions established by this handover.

Run the delivered project

Start the Node application from the delivered project directory.

Requirements and credentials

The build and tests in this handover ran with Node 26.0.0 and npm 11.12.1 on macOS. Other runtime versions were not revalidated for this handover. Internet access is needed for dependency installation, first-time MiniLM assets and live Gemini/research calls. Python 3 is used only by prompt/report regeneration helpers.

Main application setup

Unzip the package, enter Yuzee-AI-Lab, run `npm ci`, copy `.env.example` to `.env`, and set `GEMINI_API_KEY` to the receiving team's own key. Set `ADMIN_USERNAME`, `ADMIN_PASSWORD` and `AUTH_SECRET` to local values. Leave `DATABASE_URL` unset for the local file store. Run `npm run dev` and open `http://localhost:3000/`.

Build and start

Run `npm run build` to compile and type-check. For the compiled server use `NODE_ENV=production` `npm start`. Keep the delivered source prompt/schema files beside `dist`: the runtime reads them from the project directory. The source archive deliberately omits `dist` and `node_modules`; rebuild them after extraction.

Offline and model access

Without a key, the app can show its deterministic offline path; that is not a live AI verification. Model IDs are configured in `src/data/models.ts` and `RESEARCH_MODEL` in the environment. Confirm access for the receiving account; a passing local build does not confirm provider entitlement or every listed model. Never put the key in frontend code.

Separate concept and Java backend

The optional `concepts/microtools` app uses a separate Vite UI on port 3001 and optional Gemini adapter on port 3002. Its microtool result cards use fixtures. Exact commands are in its README. The Spring backend is included as source, but the new Node-side changes were not ported or retested in Java. It is not required for the main UI.

Reading order

Start with `START_HERE.md`, then `HANDOVER/AI_HANDOVER.md` and this PDF. Inspect `output/html/Yuzee-Tool-Review-Validation.html` for ten before/after examples and `output/html/Yuzee-103-Tool-Logic-Review.html` for individual tool changes. Older reports are historical context; the current v4 counselling guidance takes precedence.

Validation and its limits

Fresh checks passed for the delivered implementation. Scope matters.

Checks rerun on 16 September

npm run test:ux: 35 experience checks plus local persistence. npm run test:details: 36 scoped research checks. npm run test:output: 85 display/scenario checks. npm run test:oala: 10 integration checks. npm run test:routing: 26 routing integration checks.

Additional checks passed

npm run test:routing-tokens checked actual 256/257 token boundaries and all 103 descriptors with the locally cached tokenizer. npm run test:learning-depth checked guidance, rendering and explanation review. npm run test:tool-reviews checked all 103 reviewed snippets. npm run test:counselling checked shared flexible guidance and preserved application state. npm run build passed.

Fresh-machine tokenizer prerequisite

The token-budget test intentionally uses `local_files_only`. The archive excludes the model cache. Before running that test on a new machine, download the public tokenizer once using the command in `AI_HANDOVER.md`. Normal browser MiniLM warmup has a separate browser cache; it does not prepare this Node test cache.

Recorded live examples

The included synthetic records preserve two ten-case counselling rounds and two focused rechecks, plus earlier comparisons and selected Oala records. They are historical live Gemini results, not new provider calls made during packaging. The ten-case skill checks supplied controlled route choices, so they do not establish MiniLM ranking accuracy over all 103 tasks.

What passing tests do not prove

Schema validation is not factual correctness. Shared prompt checks are not 103 independent model-quality certifications. A content reviewer can miss unsupported assumptions. Wider semantic/routing evaluation, verified domain sources, accessibility and intended-reader testing remain necessary. Earlier recorded samples still contain wording issues documented in their review gallery.

Build and portability notes

The current build reports large JavaScript/WebAssembly assets, especially the browser embedding runtime. This is a performance concern, not a build failure. The clean-package installation and startup result is recorded separately in `HANDOVER/PACKAGE-CHECKS.json`. No new live Gemini generation is needed to verify that source packaging works.

Guide for the receiving AI

Use the existing contracts and current code as the source of truth.

Trace the main flow

Start at `src/context/TokenLabContext.tsx` and `server.ts`. Request assembly is in `src/services/YuzeeRequestAssembler.ts`; retained dialogue is managed by `TokenBudgetMemoryManager.ts` and `MultiTurnRequestBuilder.ts`. `src/oala` contains invocation, knowledge and fixed basic replies.

Routing and prompt sources

`src/routing/policy.ts` defines selection boundaries; `src/services/MicroToolRouter.ts` and `src/workers/embedder.worker.ts` implement browser routing. `microtools.json` contains runtime snippets. `counsellingStyle.json`, `learningProfiles.json`, `learningContracts.json` and `learningDepth.ts` define shared and task-specific teaching guidance.

Editing and regeneration

Individual source reviews are in `docs/plans/microtool-individual-reviews.txt`. After editing them, run `python3 scripts/build-tool-reviews.py`. After shared style/profile edits, run `python3 tmp/depth/build-guidance.py`. Inspect generated diffs and rerun relevant tests. Restart the server after prompt/runtime edits. Do not silently restore old fixed-section or never-shorten instructions.

Output, review and research

`ProtocolV13Renderer.tsx`, `ProtocolInteraction.tsx` and `chat-experience.css` own the main presentation. `ChatStreamingStatus.tsx` and `src/ux/streamProgress.ts` own status feedback. `TeachingAnswerReview.ts` edits explanation blocks only. `MoreDetails.tsx`, `ResearchAnswerCard.tsx` and `src/research` own the separate evidence workflow.

Preserve product boundaries

Keep the main prompt file and canonical schema intact unless the product owner explicitly approves a versioned change. Preserve user-entered location, explicit `@Oala` activation, controlled service actions, server-owned prompts, cancellation, missing-evidence states and interaction validation. Preserve the separate concept; do not assume its fixtures are production data.

When continuing work

First reproduce the local setup and tests, inspect the current implementation, and distinguish requirements from examples in reference files. Add focused acceptance cases for any new behaviour. Do not claim that a client-supplied routing score proves correct classification: server validation currently checks shape/bounds, not a fresh server-side embedding. Summarize actual changes and unresolved limits.

Remaining work and acceptance

A practical next backlog, with observable success criteria.

Context-aware skill recommendations

Not implemented. MiniLM currently reads only the current addressed text. A future router should use bounded, purpose-built context for references such as "Will this fit around my children?" and test corrections, topic changes and ambiguous targets. Preserve the agreed @Oala boundary unless the owner explicitly changes it.

Grounded task execution

Not implemented as a general 103-tool executor. Define per-tool required inputs, typed evidence and output contracts before connecting provider records or actions. Next-tool strings in the registry are descriptive, not a safe executable workflow. Reuse the separate research path only with explicit scope, evidence mapping and error handling.

Quality evaluation

Build a labelled routing set with paraphrases, ambiguity, long inputs and relevant user language coverage. Review semantic quality across all task families, not only JSON validity. Run comprehension sessions with younger users, parents and adults returning to learning. Check the usefulness of examples, decisions and follow-up questions.

Local preview to deployment

The handover is a local POC. Before external deployment, replace demo authentication defaults, review access and storage isolation, set operational limits, and measure provider failures, review latency and browser model loading. The local single-process file store is not evidence of multi-instance persistence readiness. No deployment is included here.

Minimum receiving-team smoke test

Open ordinary Quiz chat; answer a typed question and confirm context is retained. Enter @Oala and wait for ready; send a clear task request. Check loading/ambiguous fallback without blocking chat. Ask a narrow follow-up, type location, try a comparison on mobile, and cancel a request. With a key, test Explore more for evidence and missing-data states.

Definition of done for this handover

Source and supporting artifacts are delivered together, secrets and personal runtime history are excluded, the receiver has setup instructions, and implemented features are distinguished from gaps. The source manifest supports integrity checking. This handover transfers the current project; it does not certify every model answer or activate an external service.